

Rapport de Projet

Chatbot basé sur la pipeline RAG



Auteur : IBBOUR otmane
Encadrant : DINAR farid

École
SeaTech, Université de Toulon

Contents

1	Introduction	2
2	Architecture Générale	2
2.1	Préparation des Données	2
2.2	Stockage Vectoriel avec ChromaDB	2
2.3	Agent Intelligent (Agentic RAG)	2
2.4	Intégration d'une API Flask	2
3	Introduction à la Pipeline RAG	3
3.1	Qu'est-ce que RAG ?	3
3.2	Pourquoi utiliser RAG ?	3
3.3	Comparaison avec un modèle génératif classique	3
4	Recherche Sémantique et Bases de Données Vectorielles	4
4.1	Recherche Sémantique : Principe et Fonctionnement	4
4.2	Rôle des Bases de Données Vectorielles	5
4.3	Limites de la Recherche Sémantique	6
5	Proposition de Solution : Routing Chunks	6
5.1	Problème avec la Recherche Sémantique	6
5.2	Solution : Routing Chunks	7
6	Création de l'Agent Intelligent pour Gérer Routing Chunks et Recherche Sémantique	8
6.1	Pourquoi un Agent Intelligent ?	8
6.2	Fonctionnement de l'Agent	8
7	Développement du Frontend et Technologies Utilisées	9
7.1	Développement du Frontend avec Angular	9
7.2	Technologies Utilisées dans le Projet	10

1 Introduction

Dans ce projet, nous avons développé un chatbot intelligent basé sur l'approche Retrieval-Augmented Generation (RAG). L'objectif est d'améliorer la pertinence des réponses en combinant un modèle de génération avec une base de connaissances mise à jour.

2 Architecture Générale

L'architecture du chatbot repose sur plusieurs étapes clés :

2.1 Préparation des Données

La première étape du projet consiste à préparer les données nécessaires à l'entraînement du chatbot. Nous avons utilisé la technique de **data scraping** pour extraire automatiquement les informations depuis le site officiel de l'école. Ensuite, un **processus de nettoyage des données (Data Cleaning)** a été effectué pour supprimer les doublons, corriger les erreurs de format et structurer les informations. Enfin, les données ont été **stockées sous format JSON**, ce qui permet une indexation efficace et une exploitation optimale pour la recherche d'informations.

2.2 Stockage Vectoriel avec ChromaDB

Une fois les données structurées, elles sont converties en embeddings et stockées dans une **base de données vectorielle (ChromaDB)**. Ce stockage permet d'effectuer des recherches rapides et précises grâce à la mesure de similarité entre les embeddings des requêtes et ceux des documents.

2.3 Agent Intelligent (Agentic RAG)

L'agent intelligent joue un rôle clé dans l'optimisation des réponses. Il suit une approche dynamique de récupération des documents :

- **Routing Chunks** : Utilisé lorsque la requête appartient à une catégorie spécifique (ex: Admission, Formation). Seuls les documents pertinents sont récupérés.
- **Recherche Sémantique** : Si la requête n'est pas classifiable ou si les documents récupérés sont insuffisants, on utilise une recherche basée sur les embeddings pour compléter les informations.

2.4 Intégration d'une API Flask

Une **API Flask** a été développée pour exposer le chatbot sous forme de service web. Elle permet aux utilisateurs d'interagir avec le modèle via une interface frontend.

3 Introduction à la Pipeline RAG

3.1 Qu'est-ce que RAG ?

Retrieval-Augmented Generation (**RAG**) est une méthode qui combine la **recherche d'informations** et la **génération de texte** afin de produire des réponses plus précises et contextuelles. Contrairement aux modèles de génération classiques qui s'appuient uniquement sur leur entraînement, RAG permet d'accéder à une base documentaire externe pour enrichir ses réponses.

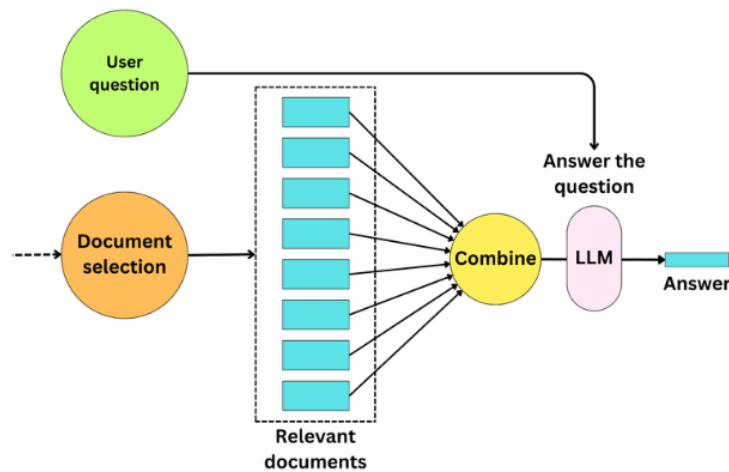


Figure 1: Schéma de la pipeline RAG

3.2 Pourquoi utiliser RAG ?

L'utilisation de RAG présente plusieurs avantages majeurs. Tout d'abord, il permet une **amélioration de la précision** en récupérant les informations les plus pertinentes avant de générer une réponse. Ensuite, il garantit l'**utilisation de données à jour et spécifiques au domaine**, contrairement aux modèles statiques qui ne peuvent pas s'adapter aux nouvelles informations. Enfin, il contribue à la **réduction des hallucinations des modèles génératifs** en s'appuyant sur des sources vérifiées, évitant ainsi la génération de contenus erronés ou fictifs.

3.3 Comparaison avec un modèle génératif classique

Un modèle de génération classique, comme GPT, produit ses réponses en s'appuyant uniquement sur les connaissances acquises lors de son entraînement, ce qui présente certaines limites. Tout d'abord, ces modèles fonctionnent avec des **données statiques**, les empêchant d'accéder à des informations récentes ou

spécifiques. De plus, ils sont sujets au **risque d'hallucination**, pouvant inventer des faits en l'absence de sources externes vérifiables. Enfin, leur **manque de contexte précis** limite leur capacité à s'adapter aux domaines spécialisés sans entraînement supplémentaire.

À l'inverse, RAG agit comme un **bibliothécaire intelligent** : il **commence par rechercher les documents les plus pertinents**, puis les utilise pour **générer une réponse contextuelle**.

Imaginez un bibliothécaire (RAG) qui cherche d'abord un livre avec la meilleure réponse avant de vous donner un résumé clair, au lieu d'un élève qui essaie d'inventer une réponse de mémoire.

Grâce à cette approche, RAG améliore la fiabilité et l'exactitude des réponses, rendant ainsi son utilisation particulièrement intéressante pour les chatbots et assistants intelligents.

4 Recherche Sémantique et Bases de Données Vectorielles

4.1 Recherche Sémantique : Principe et Fonctionnement

La recherche sémantique permet d'identifier les documents les plus pertinents pour répondre à une question en comparant leur signification plutôt que de s'appuyer sur des mots-clés exacts. Cette approche repose sur l'utilisation des **embeddings**, des représentations numériques des textes qui capturent leur signification.

Dans ce processus, chaque document et chaque requête utilisateur sont transformés en vecteurs grâce à un modèle d'embeddings. Ensuite, une base de données vectorielle permet d'effectuer une recherche efficace en mesurant la **distance cosinus** ou la **distance euclidienne** entre la requête et les documents stockés.

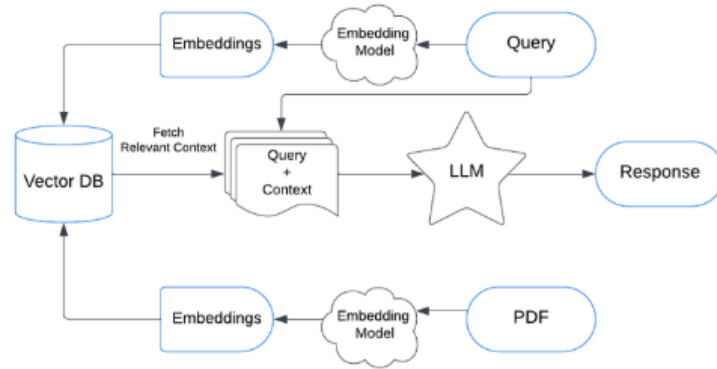


Figure 2: Schéma du fonctionnement de la recherche sémantique

4.2 Rôle des Bases de Données Vectorielles

Les bases de données vectorielles, telles que **ChromaDB**, **FAISS**, ou **Weaviate**, jouent un rôle fondamental dans la recherche sémantique. Contrairement aux bases relationnelles classiques, elles permettent d'effectuer des requêtes basées sur la similarité entre vecteurs, facilitant ainsi l'identification des documents les plus pertinents.

Ce fonctionnement repose sur trois étapes principales :

- **Indexation** : Conversion des documents en vecteurs et stockage dans la base.
- **Requête** : Transformation de la question utilisateur en un vecteur comparable.
- **Recherche par proximité** : Mesure de la distance entre la requête et les documents indexés pour identifier les plus proches.

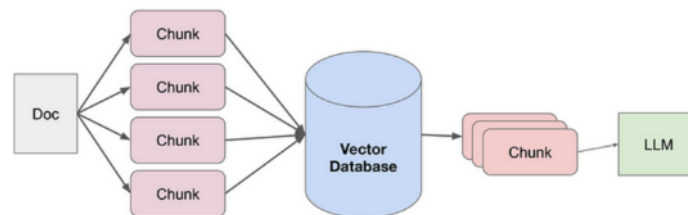


Figure 3: Illustration du stockage et de la recherche dans une base de données vectorielle

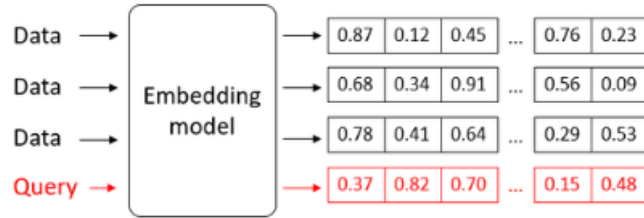


Figure 4: la représentation de données dans une base de données vectorielle

4.3 Limites de la Recherche Sémantique

Bien que la recherche sémantique soit une approche puissante, elle présente certaines limites :

- **Capacité du modèle limitée** : La taille du contexte pris en compte par le modèle (ex : 4000 tokens) peut restreindre la quantité d'informations exploitables.
- **Perte d'informations** : Si trop de documents sont récupérés, certains contenus essentiels peuvent être ignorés.
- **Inclusion de documents non pertinents** : L'algorithme peut parfois associer des documents moins pertinents en raison de la proximité des vecteurs.

L'objectif est donc d'améliorer cette approche en combinant la recherche sémantique avec des méthodes plus avancées, comme le Routing Chunks, pour optimiser la sélection des documents.

5 Proposition de Solution : Routing Chunks

5.1 Problème avec la Recherche Sémantique

La recherche sémantique pose plusieurs limites lorsqu'elle est appliquée aux sujets **Admission** et **Formation**, car ces deux catégories regroupent un grand nombre de documents dans la base de données. Plusieurs erreurs peuvent ainsi se produire :

- **Passage de documents non pertinents** : Certains documents sélectionnés peuvent ne pas être directement liés à la requête, entraînant du bruit dans la réponse.
- **Capacité limitée du modèle** : La plupart des modèles de génération (ex: LLaMA, GPT) ont une limite de **tokens** (ex: 4000), ce qui signifie que si trop de documents sont récupérés, une partie des informations peut être ignorée.

- **Réponse incomplète pour certaines questions** : En raison de la grande quantité d'informations disponibles sur l'admission et la formation, le modèle peut ne pas intégrer toutes les informations essentielles dans sa réponse.

Un exemple typique de ce problème est la question suivante : "*Quelles sont les formations disponibles à SeaTech ?*" Le mot-clé "**formation**" peut apparaître dans des contextes différents, comme dans des documents traitant des *stages de formation* ou d'autres événements académiques, ce qui peut entraîner la récupération de documents hors contexte.

Ce problème entraîne deux conséquences majeures :

- **Perte d'informations pertinentes**, car des documents essentiels peuvent être ignorés au profit de documents moins utiles.
- **Génération d'une réponse incomplète**, car le modèle peut ne pas disposer de suffisamment d'informations exploitables pour répondre précisément à la question.

5.2 Solution : Routing Chunks

Pour pallier ces limitations, nous avons mis en place une approche appelée **Routing Chunks**, qui améliore la récupération des documents en classifiant d'abord les questions avant d'effectuer la recherche. Cette approche suit plusieurs étapes :

- **Classification de la question** : Identification du sujet principal de la requête (*Admission, Formation, Autre*).
- **Filtrage des documents** : Seuls les documents appartenant à la catégorie identifiée sont sélectionnés, limitant ainsi l'inclusion de documents hors contexte.
- **Recherche sémantique complémentaire** : Si la classification ne permet pas d'obtenir une réponse satisfaisante, une recherche sémantique est utilisée en complément.

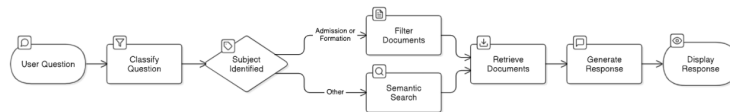


Figure 5: Schéma du Routing Chunks

L'utilisation de cette approche permet :

- **D'éviter la surcharge du modèle** avec des documents non pertinents.

- D'**améliorer la précision** en filtrant uniquement les documents utiles.
- De **réduire le risque de réponses incomplètes** en optimisant la sélection des informations avant l'envoi au modèle génératif.

Grâce à cette optimisation, nous garantissons une meilleure **qualité des réponses générées** et une utilisation plus efficace des **tokens** alloués au modèle.

6 Création de l'Agent Intelligent pour Gérer Routing Chunks et Recherche Sémantique

6.1 Pourquoi un Agent Intelligent ?

Pour garantir une récupération efficace des documents et optimiser les réponses du chatbot, nous avons conçu un **Agent Intelligent** basé sur le concept d'**Agentic RAG**. Cet agent est capable de **choisir dynamiquement** la meilleure méthode pour récupérer les documents en fonction de la question posée.

- Il **ne se limite pas à une seule approche** et peut combiner plusieurs stratégies de récupération.
- Il **adapte la stratégie** en fonction du contexte et du type de question.
- Il **privilégie le Routing Chunks** lorsque c'est possible et bascule sur la Recherche Sémantique si nécessaire.

6.2 Fonctionnement de l'Agent

L'agent suit les étapes suivantes pour récupérer et sélectionner les documents pertinents :

1. **Classification de la question** : L'agent identifie le sujet principal de la question posée (*Admission, Formation, ou Autre*).
2. **Utilisation des Routing Chunks en priorité** : Si la question concerne les sujets Admission ou Formation, seuls les documents filtrés par ces catégories sont utilisés.
3. **Passage à la Recherche Sémantique si nécessaire** : Si la question appartient à la catégorie "Autre" ou si les Routing Chunks ne fournissent pas suffisamment de documents, la recherche sémantique est activée pour récupérer des informations complémentaires.
4. **Fusion des résultats et génération de réponse** : Les documents sélectionnés sont envoyés au modèle génératif (ex : LLaMA, GPT) pour produire une réponse optimisée.

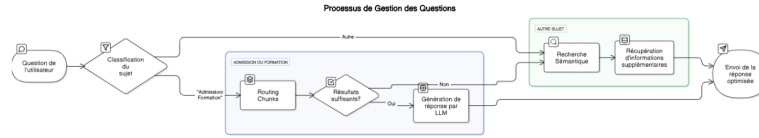


Figure 6: Schéma de l'Agent Intelligent pour gérer Routing Chunks et Recherche Sémantique

Grâce à cette approche, l'agent optimise la récupération des documents, minimise le risque d'hallucination du modèle et garantit des réponses plus complètes et précises.

7 Développement du Frontend et Technologies Utilisées

7.1 Développement du Frontend avec Angular

Pour améliorer l'expérience utilisateur et rendre l'interaction avec le chatbot plus intuitive, nous avons développé une interface frontend en utilisant **Angular**. Ce frontend permet aux utilisateurs de poser des questions, d'afficher l'historique des conversations et de voir les questions les plus fréquentes.

L'interface comprend plusieurs fonctionnalités :

- Une zone de chat interactive permettant à l'utilisateur d'échanger avec l'agent intelligent.
- Un historique des conversations pour suivre les échanges précédents.
- Une section de questions suggérées facilitant la navigation vers les informations les plus courantes.
- Une intégration fluide avec l'API Flask du backend pour récupérer les réponses du modèle RAG.

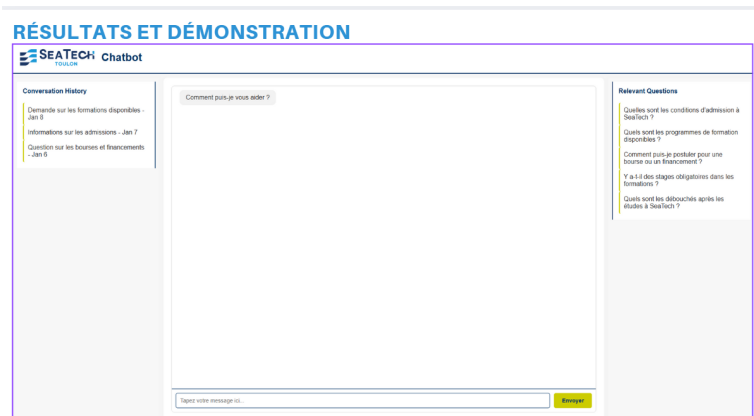


Figure 7: Interface du Chatbot développée avec Angular

7.2 Technologies Utilisées dans le Projet

Le projet repose sur une architecture combinant plusieurs technologies pour assurer une récupération efficace des informations et une interaction fluide avec l'utilisateur.

- **LangChain** : Permet d'orchestrer les différentes étapes de la récupération des documents et de l'intégration avec les modèles d'IA.
- **ChromaDB** : Base de données vectorielle utilisée pour stocker les embeddings et faciliter la recherche sémantique.
- **LLaMA 3 et GPT** : Modèles de génération de texte utilisés pour produire des réponses pertinentes.
- **Angular** : Framework frontend permettant la création d'une interface utilisateur dynamique et réactive.
- **Flask** : Framework backend léger utilisé pour gérer l'API entre l'agent intelligent et l'interface utilisateur.

L'intégration de ces technologies permet d'optimiser la récupération des informations et d'assurer une interaction fluide et performante avec le chatbot.